

TRƯỜNG ĐẠI HỌC NGUYỄN TẤT THÀNH
KHOA CÔNG NGHỆ THÔNG TIN

Bài giảng môn học: HỆ ĐIỀU HÀNH

Chương 6: **Quản lý lưu trữ**

Số tín chỉ: 3
Số tiết: 60 tiết
(30 LT + 30 TH)

Biên soạn: ThS. Nguyễn Văn Thành
Email : nvanthanh@ntt.edu.vn

Bài 6: Quản lý lưu trữ

Các thiết bị lưu trữ

Phương pháp truy xuất dữ liệu

Tổ chức lưu trữ trên đĩa cứng

Quy trình khởi động Hệ điều hành

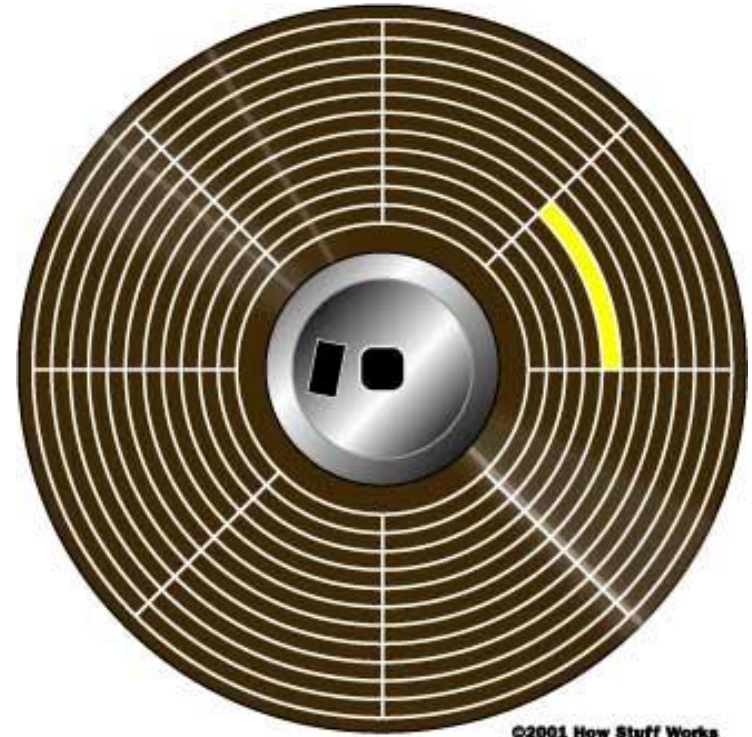
Giải thuật lập lịch truy cập đĩa cứng

Công nghệ RAID

Thiết bị lưu trữ (*Storage Device*)

■ Floppy Disk (FD) - đĩa mềm:

- ▶ Là loại đĩa từ - đĩa phủ các hạt từ tính (nam châm)
- ▶ Mỗi hạt từ lưu trữ 1 bit.
- ▶ Tổ chức lưu trữ dữ liệu:
 - *Side*: mặt đĩa (FD có **2 sides**)
 - *Track*: vòng tròn các hạt từ.
 - *Sector*: cung từ (*track* được chia nhỏ)
 - Mỗi *Sector* chứa **512 Bytes**
- ▶ Đĩa mềm 1.44 MB
 - Side=**2**, *track*=**80**, sectors/track=**18**
 - $2 \times 80 \times 18 \times 512 = 1.474.560 \text{ (B)}$

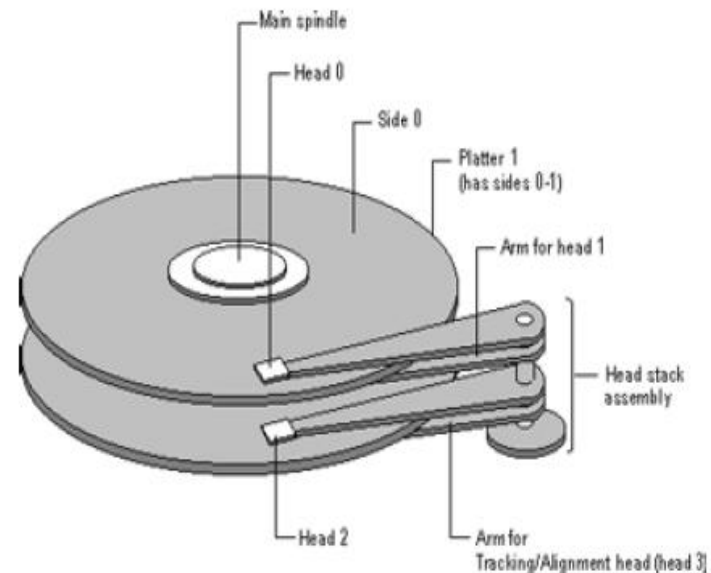


©2001 How Stuff Works

Thiết bị lưu trữ (*Storage Device*)

■ Hard Disk Drive (HDD) – đĩa cứng

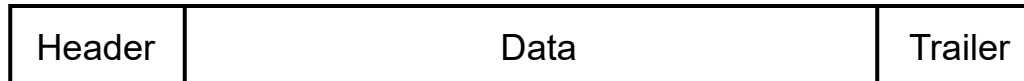
- ▶ Cấu trúc **CHS** (*Cylinder/Head/Sector*):
 - *Cylinder* (trống từ): các *track* có cùng bán kính hợp thành ống từ.
 - *Header* (đầu từ): mỗi đầu từ đọc 1 *sides*
 - *Sector* (cung từ): chia đều từ *cylinder*. **1 Sector = 512 Bytes**
- ▶ Kích thước đĩa cứng (*Size*) = **C * H * S * 512** (Bytes)



Thiết bị lưu trữ (*Storage Device*)

■ Hard Disk Drive (HDD) – đĩa cứng:

- ▶ Cấu trúc của 1 *Sector* (cung từ) trong đĩa cứng:



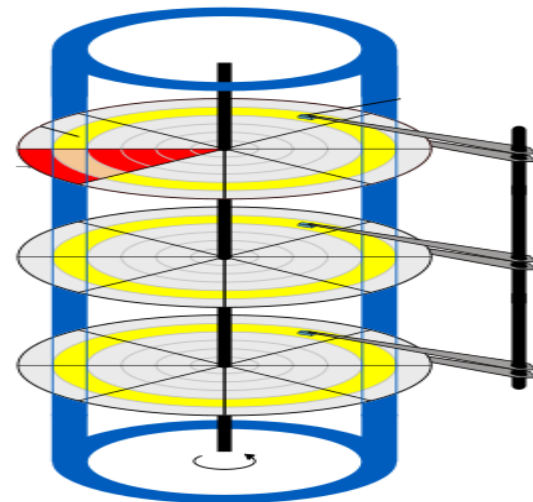
- *Header* (đầu *sector*): chứa chỉ số *sector*.
 - Data: phần chứa dữ liệu của *Sector* (512 bytes).
 - *Trailer* (đuôi *sector*): chứa mã **ECC** (*Error Correcting Code*)
- ▶ Vai trò của mã **ECC** trong Trailer:
 - Khi *Disk controller* **ghi** dữ liệu lên *sector* -> tính mã ECC của dữ liệu -> ghi vào *Trailer*.
 - Khi *Disk controller* **đọc** dữ liệu trên *sector* -> tính lại mã ECC của dữ liệu -> so sánh với mã lưu tại *Trailer*
 - Nếu so khớp nhau => xác nhận dữ liệu không bị lỗi..

Thiết bị lưu trữ (*Storage Device*)

■ Hard Disk Drive (HDD) – đĩa cứng:

▶ Đĩa cứng dùng địa chỉ **CHS**:

- Đơn vị lưu trữ trong đĩa cứng là **Sector**.
- Xác định địa chỉ 1 *Sector* trong đĩa cứng cần truy xuất bằng 3 số: địa chỉ **Cylinder**, địa chỉ **Header** và địa chỉ **Sector**



▶ Đĩa cứng dùng địa chỉ **LBA**:

- **LBA** (*Logical Block Addressing* – ấn định địa chỉ theo khối)
- Đơn vị lưu trữ trong đĩa cứng là **Block**.
- Mỗi *block* (gồm nhiều *Sector* gộp lại) được đánh số địa chỉ riêng.
- Truy cập dữ liệu trong *block* bằng *địa chỉ LBA* thay cho *địa chỉ CHS*.
- => quản lý được không gian lưu trữ lớn hơn.

Thiết bị lưu trữ (*Storage Device*)

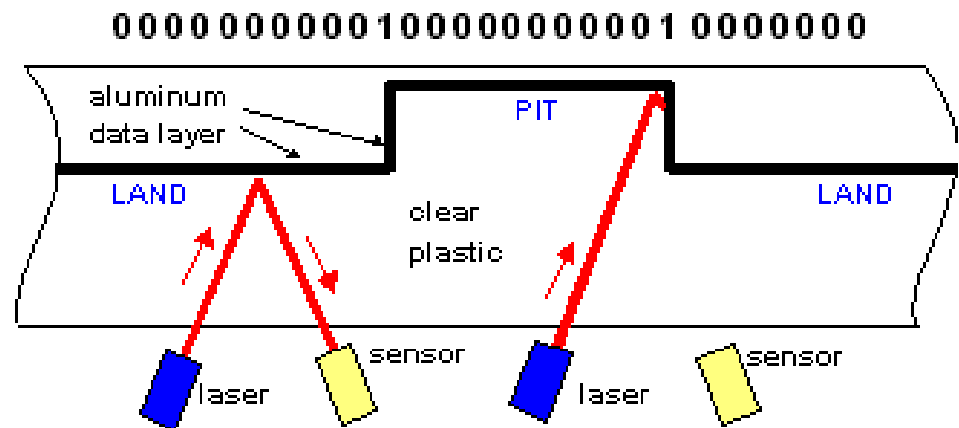
■ Optical Disc Drive (ODD) – đĩa quang:

▶ Cấu tạo:

- *Clear plastic*: lớp nhựa trong suốt.
- *Aluminum layer*: lớp nhôm phản quang.
- *Land* (mặt phẳng): phản xạ ánh sáng
- *Pit* (cái hố): không phản xạ

▶ Cách đọc dữ liệu:

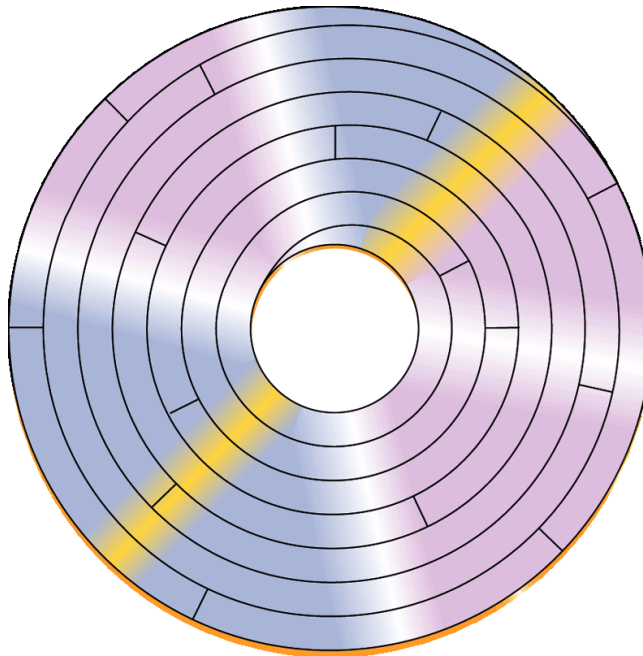
- Tia laser từ ổ chiếu vào mặt đĩa.
- Cảm biến (*sensor*) **nhận** phản xạ => **bít 1**.
- Cảm biến **không nhận** phản xạ => **bít 0**.



Phương pháp truy xuất dữ liệu

■ Truy xuất dữ liệu trên đĩa quang:

- ▶ Các track dữ liệu lưu theo hình xoắn ốc từ trong ra ngoài.
- ▶ Khi truy xuất dữ liệu: đĩa được xoay với **vận tốc tuyến tính** (*trong quay nhanh – ngoài quay chậm*).
- ▶ Nếu đọc lỗi, sẽ đọc lại ở vòng xoay kế



Phương pháp truy xuất dữ liệu

■ Truy xuất dữ liệu trên đĩa cứng (HDD):

▶ Thứ tự truy xuất (di chuyển đầu từ):

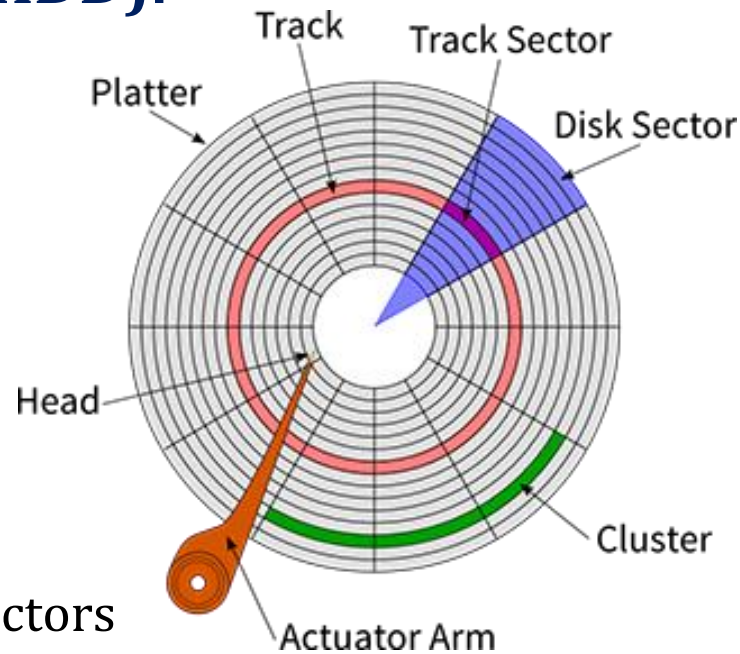
- *Cylinder* -> *Header* -> *Sector*.

▶ Vận tốc xoay đĩa:

- Số lượng *Sectors* trên mỗi *Cylinder* (từ trong ra ngoài) là bằng nhau.
- Tốc độ đọc trên mỗi *sector* là bằng nhau => **vận tốc góc cố định** (tốc độ quay như nhau khi đọc dữ liệu trên sectors bên trong hoặc ngoài)

▶ Thời gian truy xuất dữ liệu trên đĩa cứng lệ thuộc:

- *Seek time*: thời gian tìm kiếm thông tin vị trí lưu trữ dữ liệu.
- *Rotational delay*: thời gian trễ (chờ) xoay đĩa đến vị trí lưu trữ dữ liệu
- *Transfer time*: thời gian chuyển dữ liệu vào đĩa (*write*) hay chuyển dữ liệu ra khỏi đĩa (*read*).



Tổ chức lưu trữ trên đĩa cứng

■ Phân khu / phân vùng (partition):

- ▶ Để lưu trữ dữ liệu, đĩa cứng (HDD) được chia thành các phân khu (phân vùng), gọi là *partition*.



- ▶ **Windows:** mỗi *Partition* là một ổ đĩa luận lý (*Drive*).
- ▶ **Linux:** mỗi *Partition* được *mount* vào một thư mục (*Directory*).
- ▶ Quản lý dữ liệu lưu trữ trong *Partition* bằng *File System* (hệ thống quản lý tập tin)

Tổ chức lưu trữ trên đĩa cứng

■ Parttion Table:

- ▶ Là bảng ghi nhớ thông tin các phân khu được chia trên ổ cứng.
- ▶ Thông tin trong *Partition Table* cơ bản bao gồm:
 - Vị trí điểm đầu (*Starting*) và cuối (*Ending*) của *Partition* trên đĩa.
 - Dung lượng *Partition*.
 - Loại (*Type*) hệ thống quản lý tập tin sử dụng trên phân vùng.
 - Quyền được chọn khởi động Hệ điều hành (*Boot* / *Active*) của *partition*

	Type	Boot	Starting			Ending		
			Cyl	Head	Sector	Cyl	Head	Sector
1	17	00	1023	1	1	1023	254	63
2	05	00	1023	0	1	1023	254	63
3	00	00	0	0	0	0	0	0
4	00	00	0	0	0	0	0	0

Tổ chức lưu trữ trên đĩa cứng

■ Tổ chức trên đĩa cứng theo chuẩn MBR:

- ▶ **MBR** (*Master Boot Records*): là quy chuẩn tổ chức bảng phân khu (*Partition Table*) do IBM khởi xướng từ năm 1983.
- ▶ **MBR** định nghĩa 2 loại *Partition tables*:
 - *Primary Partition Table* (**PPT**) bảng phân khu chính, có *4 records* thông tin cho tối đa 4 *Primary Partition* trên đĩa cứng vật lý.
 - *Extended Partition Table* (**EPT**) bảng mở rộng, có *64 records*.
- ▶ Nếu có sử dụng *Extended Partition Table*:
 - Một *record* trên **PPT** sẽ dùng để định vị đến vị trí bảng **EPT**.
 - *Record* đầu tiên trên bảng **EPT** chứa thông tin đối chiếu với *record* trên **PPT** <= kiểm tra tính đúng đắn.
 - => **PPT** chỉ còn tối đa *3 records* cho *Primary Partition*.
 - => **EPT** chỉ còn tối đa *63 records* cho *Logical Partition*.

Tổ chức lưu trữ trên đĩa cứng

■ Tổ chức trên đĩa cứng theo chuẩn MBR:

- ▶ Các thông tin tổ chức đĩa cứng **MBR** được lưu trữ từ *Sector* đầu tiên (**Sector 0**) và kết thúc bởi 2 byte: **0x55** và **0xAA** (số *Hex*).

Master Boot Record									Extended Partition		
Partition table											
Master Boot Code	1st Partition Table Entry	2nd Partition Table Entry	3rd Partition Table Entry	4th Partition Table Entry	0x55 AA	Primary Partition (C:)	Primary Partition (E:)	Primary Partition (F:)	Logical Drive (G:)	Logical Drive (H:)	Logical Drive n

Ghi chú: trong một số tài liệu tiếng Việt cho rằng **MBR** lưu trữ tại **Sector 1** của đĩa vật lý (có lẽ do dịch từ “*the 1st sector*”).

Tổ chức lưu trữ trên đĩa cứng

■ Tổ chức trên đĩa cứng theo chuẩn GPT:

- ▶ **GPT** (*GUID Partition Table*): nhiều cải tiến nhằm thay thế **MBR**.
- ▶ So sánh **GPT** và **MBR**:

Các tính chất	MBR	GPT
Số lượng <i>Primary Partition</i> tối đa (GPT không có <i>Extended partition</i>)	4	128
Dung lượng tối đa của một Partition	2TB	2EB
Định danh toàn cục cho mỗi phân vùng (Globally Unique Identifier – GUID)	Không	Có
Cơ chế tự kiểm soát lỗi	Không	Có

- ▶ Cấu trúc khởi động của GPT:
 - ***EFI Partition***: phân vùng duy nhất được chọn khởi động.
 - Các ***partition khác***: chứa Hệ điều hành hoặc dữ liệu.

Quy trình khởi động Hệ điều hành

■ Boot loader và Bootstrap:

▶ Nhu cầu cần *Boot Loader*:

- Một phân vùng chứa rất nhiều tập tin (files).
- Máy tính không thể xác định được đâu là các tập tin chạy Hệ điều hành (*Kernel files*) để nạp vào RAM cho CPU thực thi.

▶ Định nghĩa *Boot Loader* (bộ nạp Hệ điều hành)

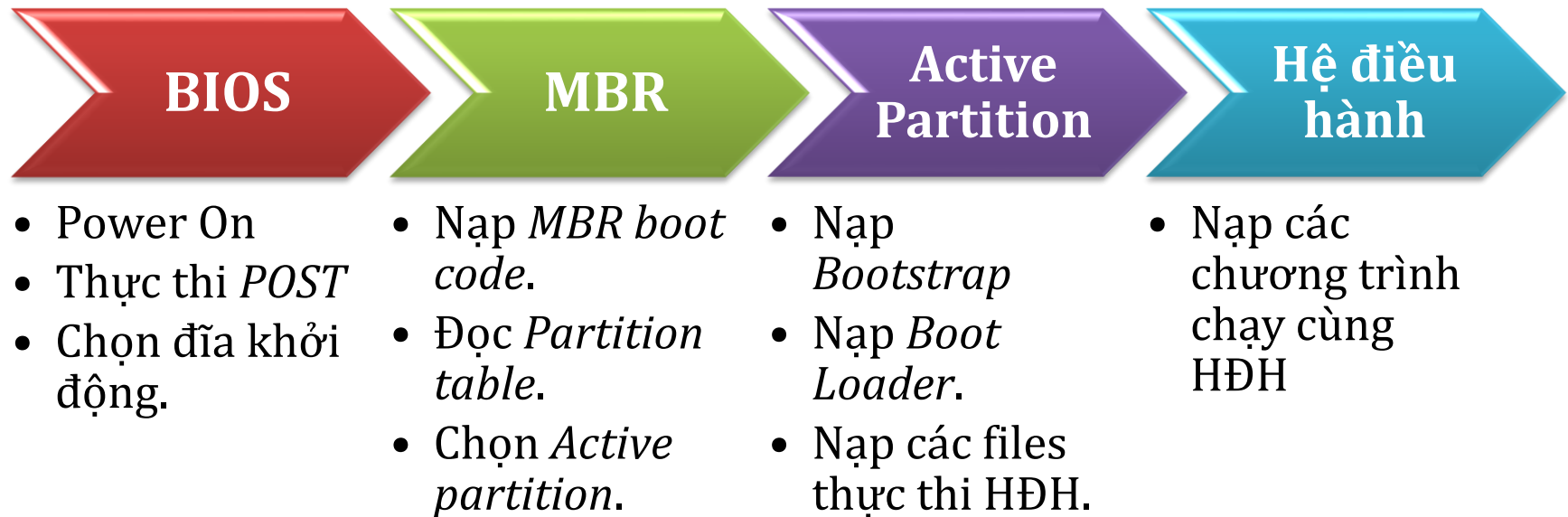
- Là phần mềm chứa các chỉ thị nạp các tập tin thực thi Hệ điều hành.
- Một *Boot Loader* có thể dùng nạp nhiều loại Hệ điều hành khác nhau.

▶ Định nghĩa *Bootstrap loader*:

- Là bộ *code* dùng nạp *Boot loader* vào RAM cho CPU thực thi.
- Bộ *code Bootstrap Loader* lưu trữ tại *Boot sector* (hay *Boot block*) của *Active Partition*.
- Một số hệ thống: *Bootstrap* được cài đặt sẵn trong **BIOS ROM**.

Quy trình khởi động Hệ điều hành

■ Quá trình tìm nạp Hệ Điều Hành:



- **POST** (*Power On Self Test*): chương trình tự kiểm tra khi *Power On*.
- **BIOS** (*Basic Input / Output System*): chương trình nạp sẵn trong ROM.
- **MBR** (*Master Boot Record*): các thông tin tổ chức đĩa cứng.
- **Bootstrap**: bộ nạp chương trình *Boot loader*.
- **Boot Loader**: chương trình nạp tập tin khởi động của Hệ điều hành.

Giải thuật lập lịch truy cập đĩa cứng

■ Dẫn nhập lập lịch truy cập đĩa cứng:

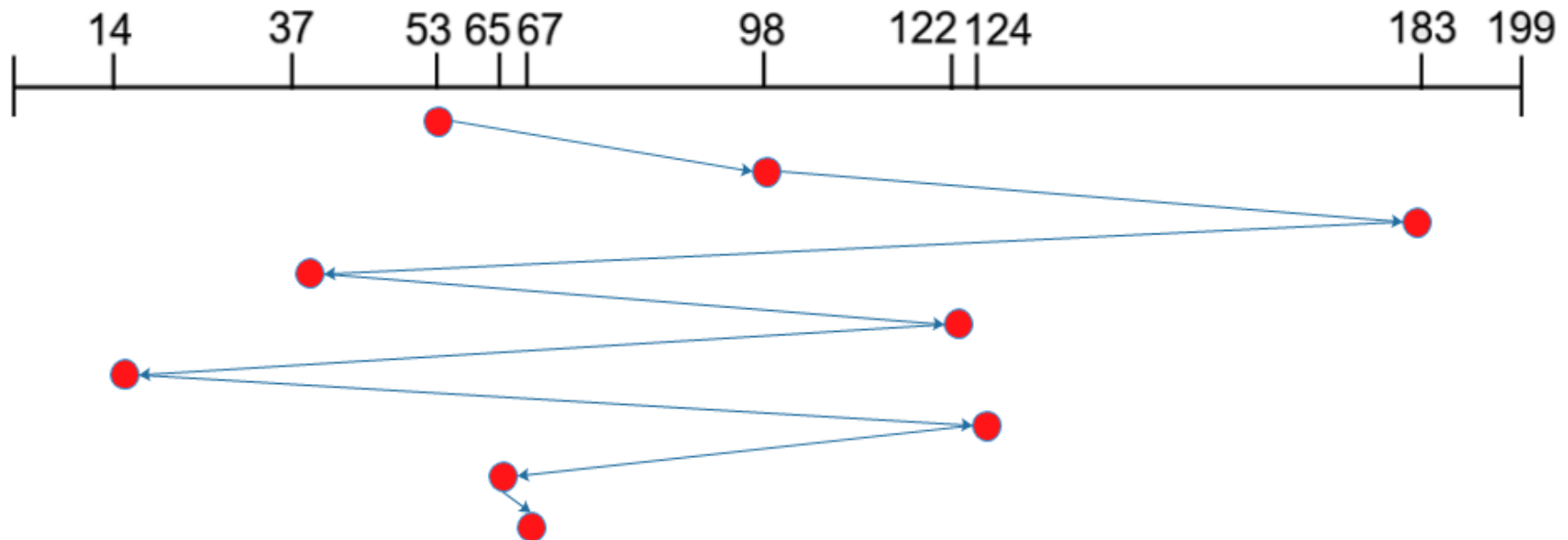
- ▶ Cơ chế tìm đọc / ghi dữ liệu trên đĩa:
 - Đầu từ di chuyển đến *Cylinder* (chứa *sector*).
 - *Motor* xoay phiên đĩa để đưa *Sectors* cần đọc / ghi tiếp cận với đầu từ.
- ▶ *Seek time*: thời gian đầu từ tiếp cận sector.
 - Tốc độ xoay đĩa là cố định (5400/7200 rpm).
 - *Seek time* lệ thuộc thời gian đầu từ di chuyển qua các *Cylinder*
- ▶ Lập lịch truy cập đĩa cứng:
 - Hàng đợi (*Disk queue*) nhận nhiều yêu cầu truy cập dữ liệu ở nhiều *Cylinder* khác nhau.
 - Giải thuật lập lịch chọn ra thứ tự các yêu cầu sẽ được đáp ứng.
 - Giải thuật được cho là tốt nhất nếu cho *Seek time* nhỏ nhất.



Giải thuật lập lịch truy cập đĩa cứng

■ Giải thuật FCFS (First Come, First Served)

- ▶ **Thuật giải**: Yêu cầu vào hàng đợi trước sẽ được đáp ứng trước.
- ▶ **Minh họa**: Đầu đọc đang ở *Cylinder* số **53**. Chuỗi yêu cầu đọc/ghi đĩa trong hàng đợi là: **98, 183, 37, 122, 14, 124, 65, 67**



Giải thuật lập lịch truy cập đĩa cứng

■ Giải thuật First Come, First Served (FCFS)

- ▶ Tổng số bước di chuyển của đầu đọc giữa các *Cylinder* là **640**.

Lần	Current	Target	Steps
1	53	98	45
2	98	183	85
3	183	37	146
4	37	122	85
5	122	14	108
6	14	124	110
7	124	65	59
8	65	67	2
Tổng số bước di chuyển:			640

Giải thuật lập lịch truy cập đĩa cứng

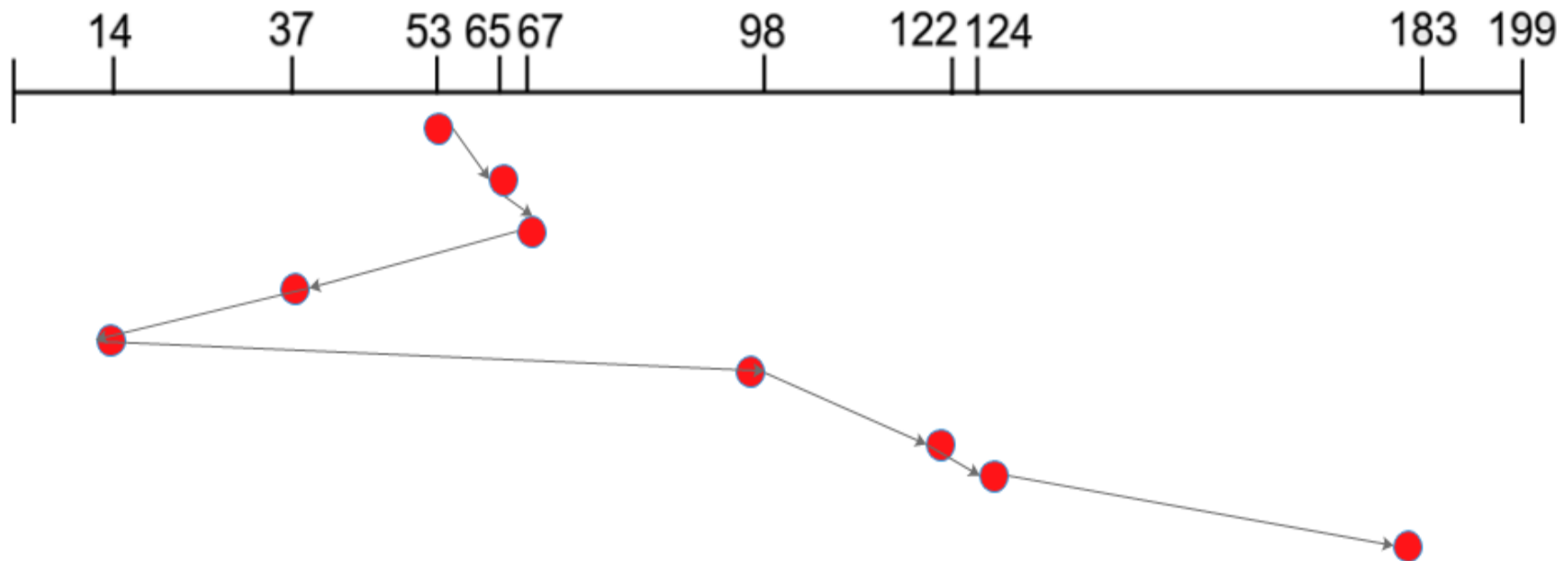
■ Giải thuật SSTF (Shortest-Seek-Time First)

▶ **Thuật giải**: yêu cầu gần vị trí hiện tại nhất sẽ được đáp ứng trước.

▶ **Minh họa**:

• Đầu đọc đang ở *Cylinder* số **53**. Chuỗi yêu cầu đọc/ghi đĩa trong hàng đợi là: **98, 183, 37, 122, 14, 124, 65, 67**

• Phục vụ: **53, 65, 67, 37, 14, 98, 122, 124, 183**



Giải thuật lập lịch truy cập đĩa cứng

■ Giải thuật SSTF (Shortest-Seek-Time First)

- ▶ Tổng số bước di chuyển của đầu đọc giữa các *Cylinder* là **236**.

Lần	Current	Target	Steps
1	53	65	12
2	65	67	2
3	67	37	30
4	37	14	23
5	14	98	84
6	98	122	24
7	122	124	2
8	124	183	59
Tổng số bước di chuyển:			236

Giải thuật lập lịch truy cập đĩa cứng

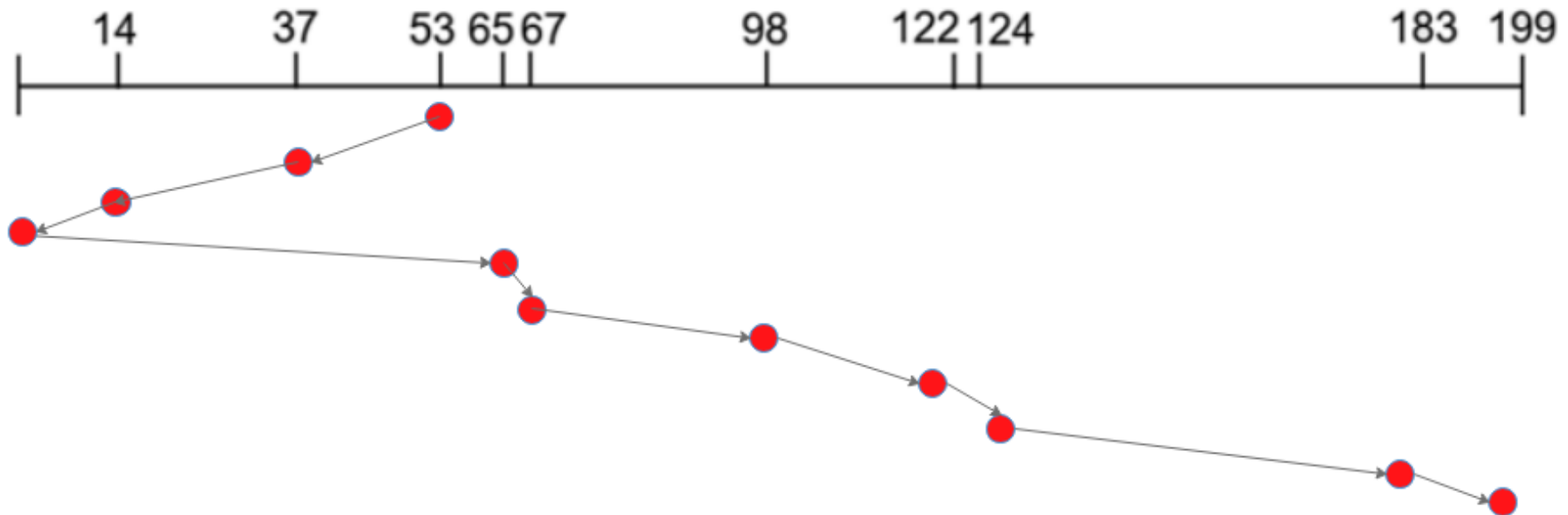
■ Giải thuật SCAN (thuật toán thang máy – elevator)

▶ Thuật giải:

- Đầu từ di chuyển 2 chiều: từ *cylinder* **0** đến *cylinder* cuối rồi ngược lại
- Trên đường đi, nếu gặp *cylinder* đáp ứng yêu cầu thì sẽ phục vụ.

▶ Minh họa: *Cylinder* số **53**, queue = **98, 183, 37, 122, 14, 124, 65, 67**

▶ Phục vụ: **53, 37, 14, 0, 65, 67, 98, 122, 124, 183, 199.**



Giải thuật lập lịch truy cập đĩa cứng

■ Giải thuật SCAN (thuật toán thang máy – elevator)

- ▶ Tổng số bước di chuyển của đầu đọc giữa các *Cylinder* là **208** (nếu bỏ qua quá trình di chuyển đến *cylinder cuối cùng* và *cylinder đầu tiên*)

Lần	Current	Target	Steps
1	53	37	16
2	37	14	23
	14	0	14
3	14	65	51
4	65	67	2
5	67	98	31
6	98	122	24
7	122	124	2
8	124	183	59
Tổng số bước di chuyển:			208
Tổng số bước di chuyển thực tế:			222

Giải thuật lập lịch truy cập đĩa cứng

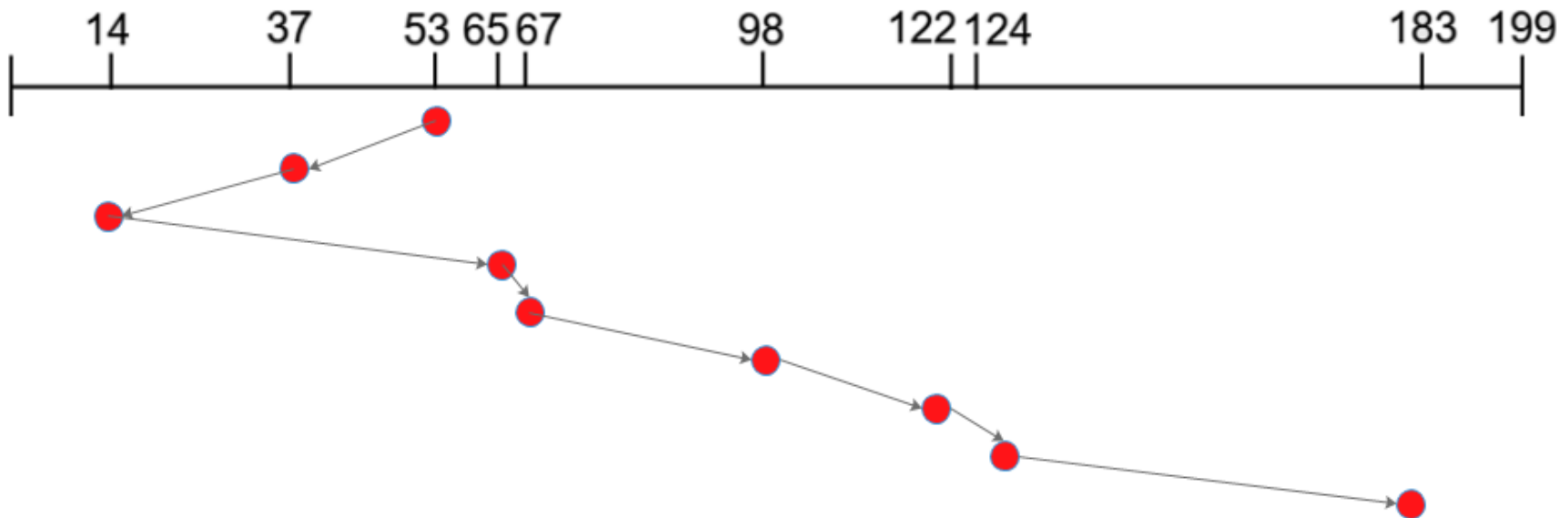
■ Giải thuật LOOK (thuật toán thang máy cải tiến):

▶ Thuật giải:

- Đầu từ di chuyển 2 chiều tương tự SCAN, nhưng sẽ đảo chiều sau khi đến từ *cylinder yêu cầu nhỏ nhất* (hoặc *lớn nhất*).

▶ Minh họa: Cylinder số **53**, queue = **98, 183, 37, 122, 14, 124, 65, 67**

▶ Phục vụ: **53, 37, 14, 65, 67, 98, 122, 124, 183**



Giải thuật lập lịch truy cập đĩa cứng

■ Giải thuật LOOK (thuật toán thang máy cải tiến)

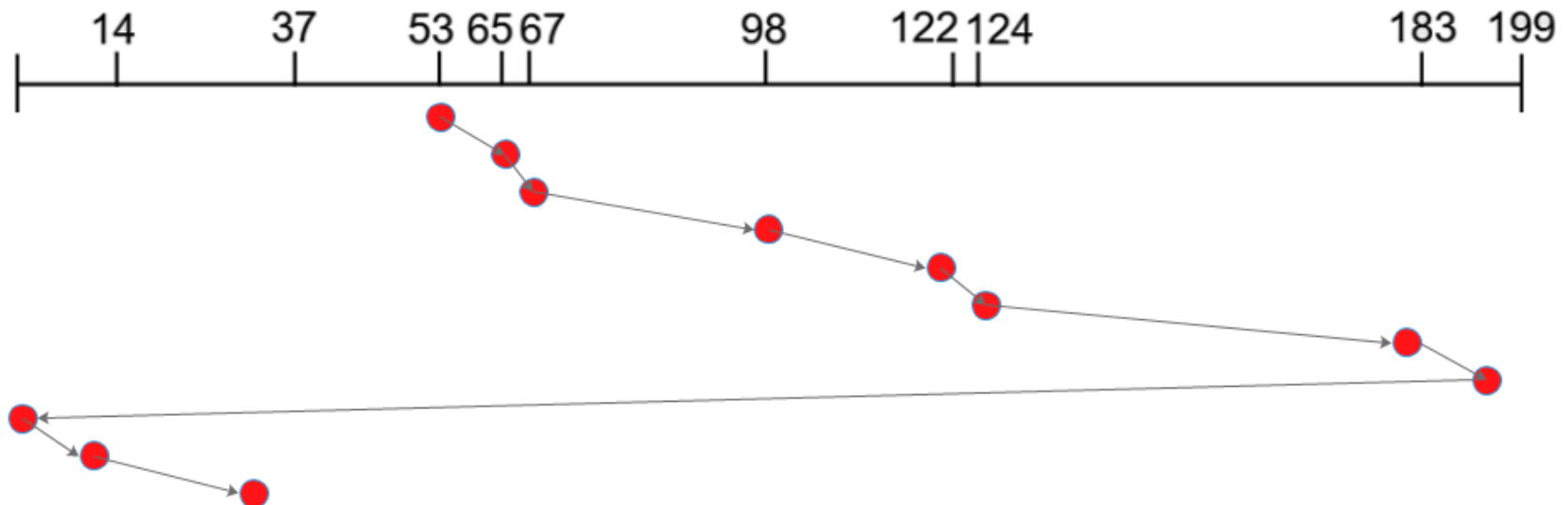
- ▶ Tổng số bước di chuyển của đầu đọc giữa các *Cylinder* là **208**.

Lần	Current	Target	Steps
1	53	37	16
2	37	14	23
3	14	65	51
4	65	67	2
5	67	98	31
6	98	122	24
7	122	124	2
8	124	183	59
Tổng số bước di chuyển:			208

Giải thuật lập lịch truy cập đĩa cứng

■ Giải thuật Circular-SCAN (C-SCAN)

- ▶ **Thuật giải:** tương tự **SCAN**, nhưng đầu đọc chỉ di chuyển 1 chiều (đến *cylinder cuối cùng* rồi quay về *cylinder đầu tiên*).
 - Trong khi di chuyển, gặp *Cylinder* đáp ứng yêu cầu sẽ phục vụ.
- ▶ **Minh họa:** *Cylinder* số **53**, queue = **98, 183, 37, 122, 14, 124, 65, 67**
- ▶ **Phục vụ:** **53, 65, 67, 98, 122, 124, 183, 199, 0, 14, 37**



Giải thuật lập lịch truy cập đĩa cứng

■ Giải thuật Circular-SCAN (C-SCAN)

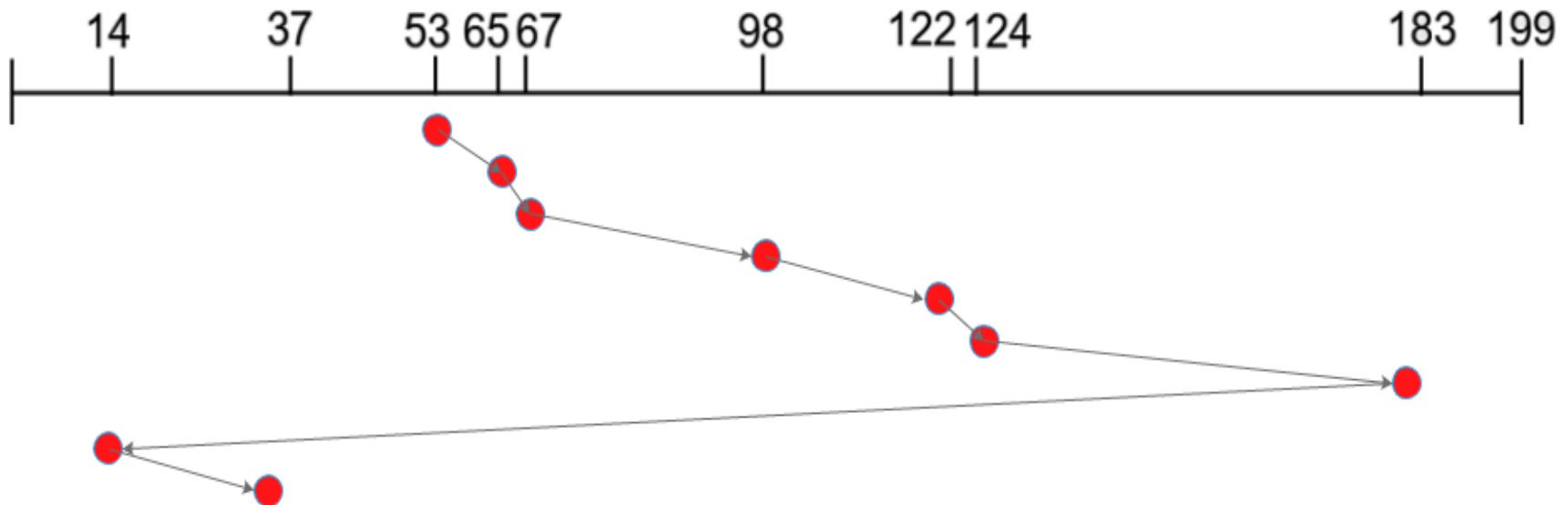
- ▶ Tổng số bước di chuyển của đầu đọc giữa các *Cylinder* là **184** (nếu xem số bước di chuyển từ *cylinder cuối cùng* về *cylinder đầu tiên* = 1)

Lần	Current	Target	Steps
1	53	65	12
2	65	67	2
3	67	98	31
4	98	122	24
5	122	124	2
6	124	183	59
	183	199	16
	199	0	1
7	0	14	14
8	14	37	23
Tổng số bước di chuyển:			184

Giải thuật lập lịch truy cập đĩa cứng

■ Giải thuật Circular-LOOK (C-LOOK)

- ▶ **Thuật giải**: tương tự C-SCAN. Nhưng đầu đọc chỉ di chuyển đến *Cylinder* yêu cầu lớn nhất rồi quay về *Cylinder* yêu cầu nhỏ nhất.
- Trong khi di chuyển, gặp *Cylinder* đáp ứng yêu cầu sẽ phục vụ.
- ▶ **Minh họa**: *Cylinder* số **53**, queue = **98, 183, 37, 122, 14, 124, 65, 67**
- ▶ **Phục vụ**: **53, 65, 67, 98, 122, 124, 183, 14, 37**



Giải thuật lập lịch truy cập đĩa cứng

■ Giải thuật Circular-LOOK (C-LOOK)

- ▶ Tổng số bước di chuyển của đầu đọc giữa các *Cylinder* là **184** (xem số bước từ **183** đến **14** = (**183** đến **199**) + (**199** đến **0**) + (**0** đến **14**) = **31**)

Lần	Current	Target	Steps
1	53	65	12
2	65	67	2
3	67	98	31
4	98	122	24
5	122	124	2
6	124	183	59
7	183	14	31
8	14	37	23
Tổng số bước di chuyển			184

Giải thuật lập lịch truy cập đĩa cứng

■ Đánh giá các giải thuật:

- ▶ **FCFS** làm cho đầu đọc phải di chuyển nhiều nhất (không tối ưu)
- ▶ **SSTF** là phổ biến và “tự nhiên”
- ▶ **SCAN** và **C-SCAN** là tốt với các hệ thống đọc ghi đĩa nhiều.
- ▶ **C-LOOK** tiết kiệm các bước di chuyển đến đầu và cuối *Cylinder* so với **C-SCAN**.
- ▶ Hiệu năng nói chung phụ thuộc vào số lượng và tính chất của các yêu cầu truy cập đĩa

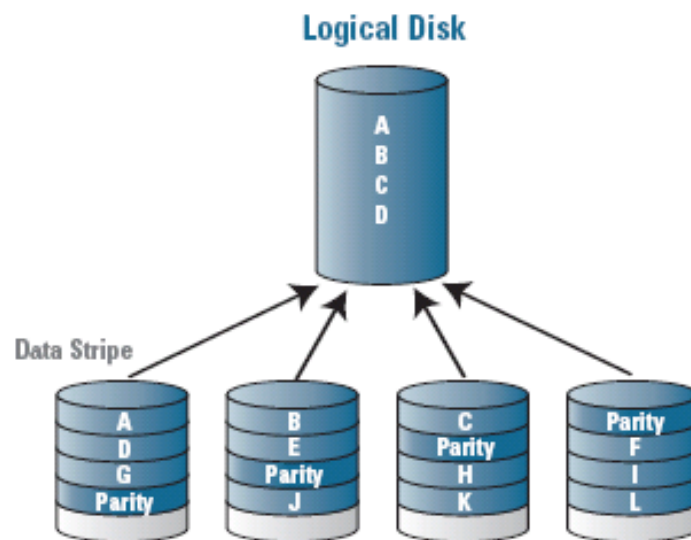
■ Cài đặt:

- ▶ Các Hệ điều hành cài đặt mỗi thuật toán lập lịch như một *module* độc lập để dễ lựa chọn khi cần thiết
- ▶ **SSTF** hoặc **C-LOOK** thường được Hệ điều hành chọn là mặc định.

Công nghệ RAID

■ Tổng quan về RAID:

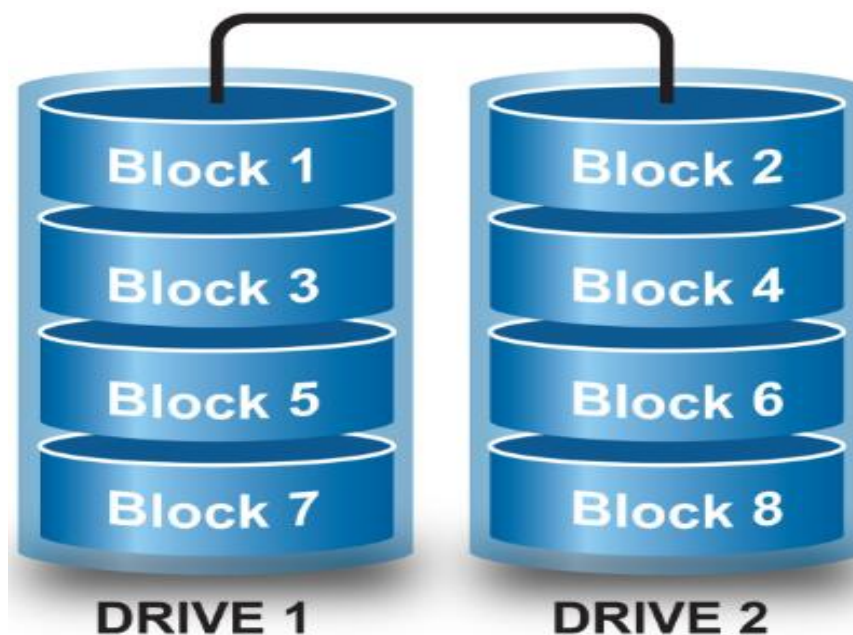
- ▶ **RAID** (*Redundant Arrays of Independent Disks*) là công nghệ phối hợp *nhiều Physical Disks* thành *một Logical Disk*.
- ▶ Mục đích của RAID:
 - Tăng tốc độ truy xuất trên *Logical Disk* (**RAID-0**)
 - Đảm bảo an toàn cho dữ liệu lưu trữ trên *Logical Disk* (**RAID-1**)
 - Hoặc kết hợp cả 2 mục đích trên (**RAID-5, RAID-10, RAID-01**)
- ▶ RAID chỉ hỗ trợ trên chuẩn giao tiếp **SATA, SCSI** và **SAS**.



Công nghệ RAID

■ RAID-0 (*Striping*):

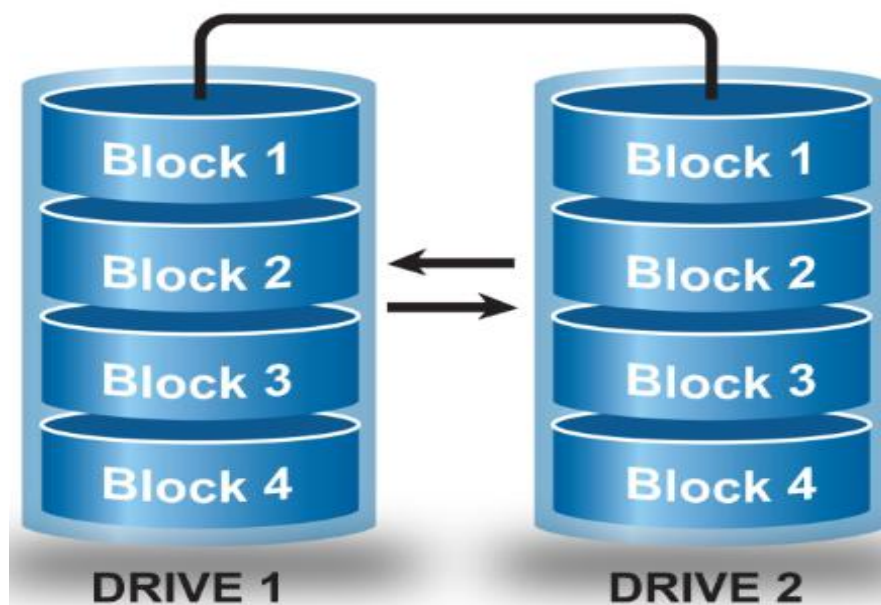
- ▶ Dữ liệu lưu trữ được chia thành nhiều Blocks.
- ▶ Các block được lưu trữ rải trên các ổ vật lý trong RAID-0
- ▶ Gia tăng tốc độ truy xuất dữ liệu.
- ▶ Dữ liệu sẽ hỏng khi 1 trong các ổ vật lý bị lỗi.



Công nghệ RAID

■ RAID-1 (*Mirroring*):

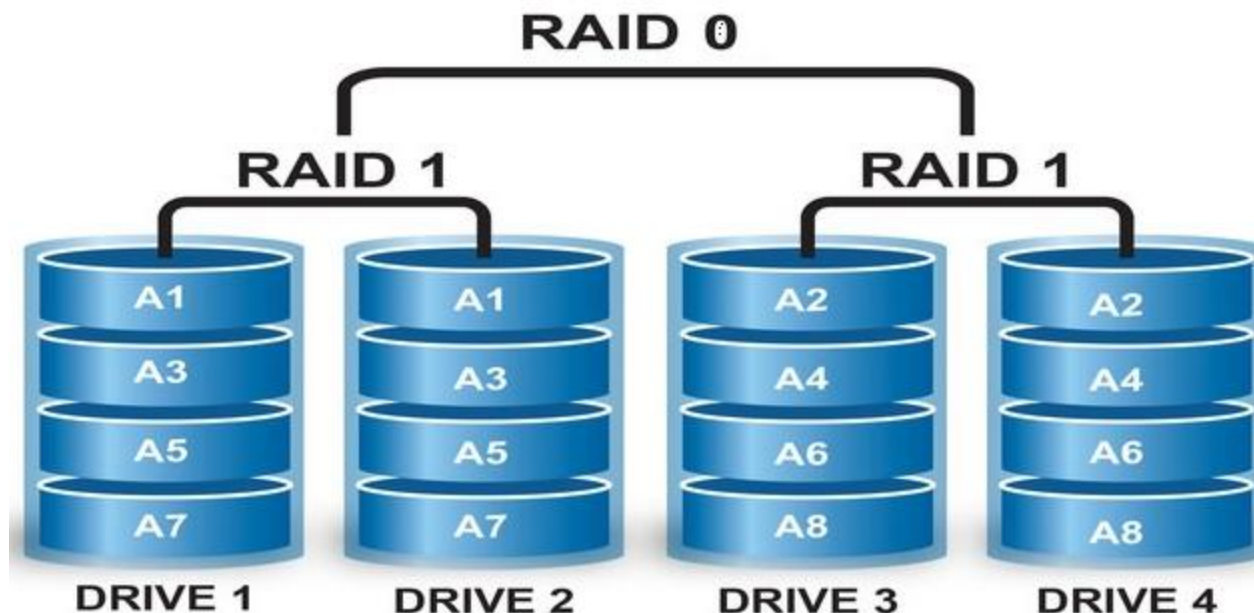
- ▶ Các Blocks dữ liệu được lưu trữ trên 1 ổ vật lý.
- ▶ Ổ vật lý còn lại sẽ lưu dự phòng toàn bộ Blocks của ổ kia..
- ▶ Tăng tính an toàn cho dữ liệu lưu trữ - Dữ liệu vẫn còn khi 1 trong các ổ vật lý bị lỗi.
- ▶ Không cải thiện tốc độ truy xuất.



Công nghệ RAID

■ RAID-10 (Kết hợp *Mirroring* và *Striping*):

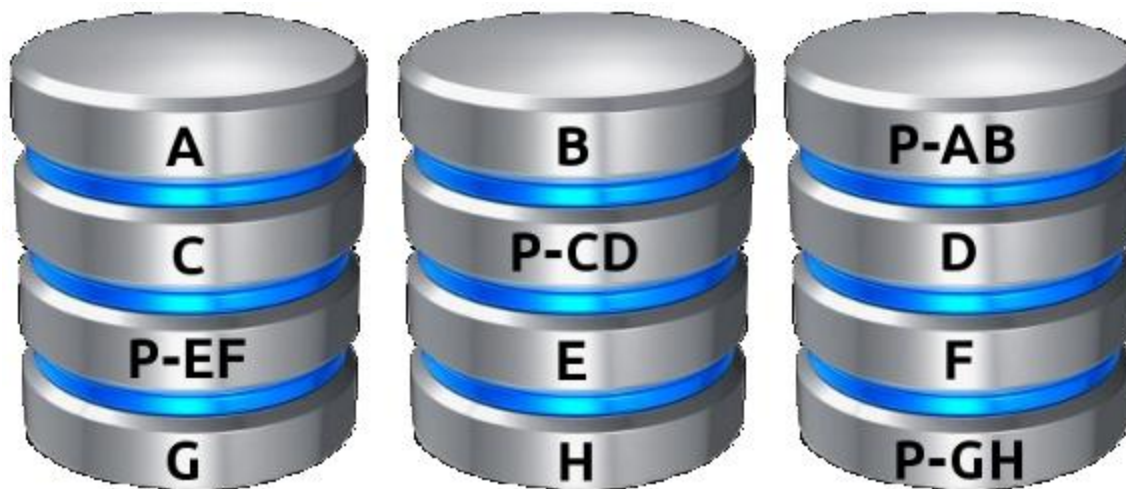
- ▶ 4 đĩa vật lý chia thành 2 cụm. Mỗi cụm chạy RAID-1 (*Mirroring*).
- ▶ Các *blocks* dữ liệu được phân đôi (RAID-0 - *Striping*) trước khi đưa vào lưu trữ trên 2 cụm RAID-1.
- ▶ Gia tăng tốc độ truy xuất dữ liệu.
- ▶ Đảm bảo an toàn dữ liệu khi 1 trong các ổ vật lý bị lỗi.



Công nghệ RAID

■ RAID-5 (*Parity*):

- ▶ Phương thức RAID tiết kiệm ổ đĩa mà vẫn đảm bảo *tính an toàn* và *tăng tốc độ* truy xuất.
- ▶ **RAID-5** dùng **n** HDD ($n \geq 3$),
- ▶ Luân phiên dùng **n-1** HDD lưu trữ các *Blocks* (*Striping*).
- ▶ Luân phiên dùng **1** HDD còn lại lưu trữ tổng các *Blocks* của n-1 HDD kia (*Mirroring*).



Cám ơn !

